



NCA Sponsored Training

Day 1 – Morning Session (Lab)

Shell 101

Ahmed Didouh

Madinah

12/01/2026



rc3.kaust.edu.sa





Outline



1. Introduction to Unix-based OS

- *What is Linux*
- *How to interact with the OS*

2. The Unix Shell

- *Introduction*
- *Basic Commands*

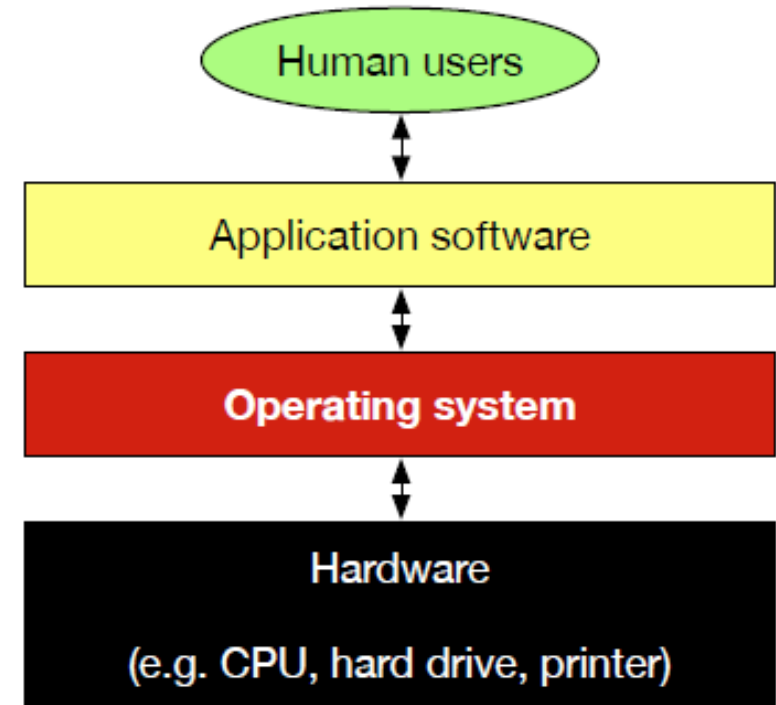


What is Linux?



Linux is an Operating System (OS)

- Software that manages the computer's hardware and provides common services for software (read keyboard input, draw window on screen, etc.)
- Include useful software utilities (compiler, text editor, etc.)





What is Linux?



The Linux OS is a combination of:

- Linux kernel
 - Provides core OS functionality (e.g. process and hardware management)
 - Multitasking and multiuser, advanced security
 - Open source, created by Linus Torvalds at the University of Helsinki in 1991
- GNU's Not Unix (GNU) system software
 - Open-source software project started in 1983 by Richard Stallman at MIT
 - Intended to become its own OS but the kernel (HURD) isn't ready yet
 - Includes glibc C library, libstdc++ C++ library, gcc C/C++ compiler, gdb debugger, coreutils, binutils, bash shell, GNOME desktop env., Emacs text editor, etc



Interacting with the operating system



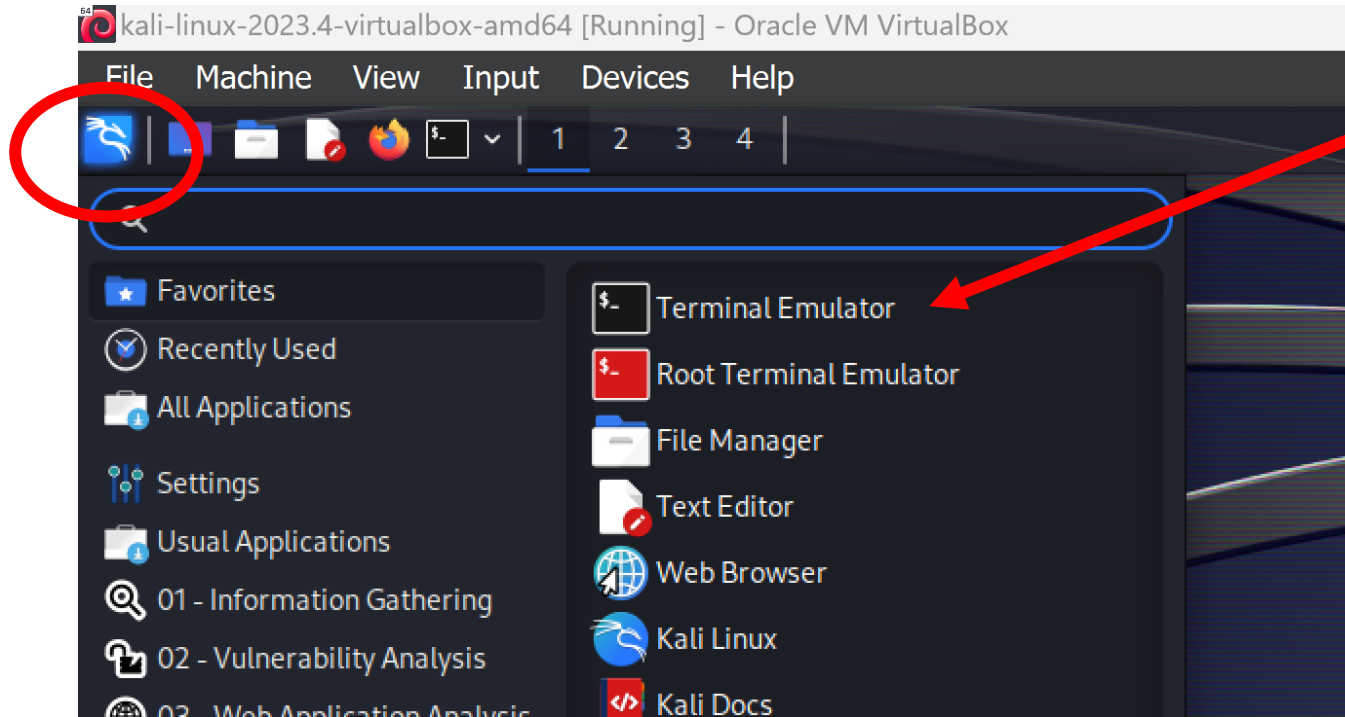
- Graphically
 - Graphical primitives such as windows, icons and buttons
 - Common window managers: KDE, GNOME, MATE (GNOME fork), XFCE
- Textually (our focus today)
 - Through a command-line interpreter or shell
 - Very powerful, can actually be seen as a programming language
 - Can run inside a graphical window (terminal emulator)
 - Common Unix shells conforming to the POSIX standard: bash, dash, csh



Our First Command



Open up a terminal emulator:





Our First Command



The Shell is waiting for a command

- Type *echo Hello World* and hit the Enter key
- What happened?

Prompt

Your Command Here

The screenshot shows a terminal window with a dark background. At the top right, the text 'kali@kali: ~' is visible. Below it is a menu bar with 'File', 'Actions', 'Edit', 'View', and 'Help'. The main area of the terminal shows a prompt '(kali@kali)-[~]' followed by a blue dollar sign '\$' and a white cursor. Two red arrows point from the text 'Prompt' and 'Your Command Here' to the '\$' and the cursor respectively.



Our First Command



What happened?

1. The command was executed, yielding the writing of Hello World onto the terminal window
2. A prompt is displayed, allowing us to enter a new command

The **echo** program writes its arguments to the standard output (which is, by default, the terminal) and a new line





Anatomy of a Command



ls -l -a /home/kali

↑
executable

↑
options

↑
arguments

Different programs support different options and arguments



Getting Help With Commands



- Every GNU program has an associated manual page
- The command *man* *<program-name>* lets us access it
- Navigating a manual page
 - We can use the navigation keys (e.g. arrow keys) to move around
 - We can search by hitting /, typing in your query and hitting entry
 - To navigate multiple hits we can use n for next and Shift+n for previous
 - Finally, we can exit the manual by hitting q



Getting Help With Commands



Exercise

- The *uname* program can be used to obtain system information
- Use *man* to figure out how to use *uname* to obtain:
 - The operating system of the computer you are working on
 - The kernel release of your Linux OS
- Use *uname* for the aforementioned purposes



Navigating command history



Retyping entire commands can be cumbersome

- We can use the **up** and **down** keys to recall **previous** commands

Exercise:

Experiment with your command history, search for the *echo Hello World* command



Some Important Things



`~` = My Home Directory

`clear` = Clear the terminal

`echo` = Print something on the screen

`[TAB]` = Auto-complete commands



Linux Filesystem



cd – change directory

pwd – print name of current/working directory

ls – list directory contents

touch <path> – will create an empty file <path>

nano <path> – is a user-friendly terminal-based text editor

mkdir <path> – creates new empty folder <path>

cat <path> – prints entire to the standard output, bad for large files

less <path> – prints a single screen's worth, then pauses



Filesystem Exercises



1. Create a folder named *scripts* inside your home folder
2. Create a file named *hello.sh* inside the *scripts* folder
3. Edit it using a text editor so that it contains the following two lines:

```
#!/bin/bash  
echo "Hello world!"
```

1. Print *hello.sh*'s contents to the standard output



File Permissions



Unix has built-in file security. Every file has these permissions

	Owner	Group	Others
Read (r)	yes or no	yes or no	yes or no
Write (w)	yes or no	yes or no	yes or no
Execute (x)	yes or no	yes or no	yes or no

We can list the permissions of a given file using `ls -l`



File Permissions



`chmod <opt> <perm> <p1> ... <pn>` – changes permissions

`perm` is the permissions string. It is highly flexible

Example:

- `chmod u=rwx,g=rx,o= hello.sh` gives, on file `hello.sh`, all permissions to owner, read and execute permissions to group and none to others
- `chmod +x hello.sh` gives execute permission to everyone
- `chmod -x hello.sh` removes execute permission for everyone



File Execution



- The shell only looks for executables in your the folders specified in the PATH environment variable
- With the prefix `./` you can execute a file that is not in your PATH but in your working directory. For executables located elsewhere we need a complete (absolute or relative) path

Exercise:

1. Give execution permissions to our hello script
2. Run our hello script



rm *<path1>* – removes regular files

mv *<path1>* ... *<pathn>* *<dst>* – move files to dst (cut and paste)

cp *<path1>* ... *<pathn>* *<dst>* – copy files to dst (copy and paste)



Redirection



Most programs read from the standard input (keyboard) and write to the standard output (terminal). We can use redirection to alter that behavior.

Redirecting output with `>` and `>>`

- redirect echo's output to a file, overwriting existing contents (if any)
 - `echo "Hello hello hello" > file.txt`
- append echo's output to a file (preserves existing contents)
 - `echo "how low" >> file.txt`

Do you have any questions?



rc3.kaust.edu.sa



Follow us @rc3kaust

